PLATTER

# What's Next: The 6 Remaining Jobs

The app works end to end and is on GitHub. This is the plan for everything left: what each job is, why it matters, the steps, the traps, and how you know it is done.

## The six jobs at a glance

| # | Job | Size | Needs first | Why it matters |
|---|---|---|---|---|
| **1** | Finish the GitHub page | Small | nothing | It is the first thing a recruiter sees |
| **2** | Deploy the app | Medium | 1 (for the link) | A live link beats any screenshot |
| **3** | Password reset by email | Medium | 2 (for real emails) | A real app cannot lock people out forever |
| **4** | Tests and CI | Large | nothing | Shows you ship code you can trust |
| **5** | Real payments (Razorpay) | Large | 2 | Shows you can handle money safely |
| **6** | Polish | Pick any | nothing | Small touches that show care |

**Remember.** **Suggested order: 1, 2, 4, 3, 5, 6.** Do the GitHub page and the deployment first, because a live link is what you put on your resume. Tests come next because they are a common interview topic and they protect everything you add after. Password reset and payments both need the live backend, so they come after deployment.

## 1. Finish the GitHub page

**Goal.** The repository looks finished and trustworthy within five seconds of opening it.

### Checklist

- [ ] Fill in the **About** box: description, topics (react, vite, tailwindcss, nodejs, express, mongodb, mongoose, jwt, rest-api, food-delivery, full-stack, portfolio-project).
- [ ] Upload a **social preview image** (Settings, General, Social preview; 1280 x 640 px). Use the home screenshot.
- [ ] Retake the home screenshot with **New Delhi** selected so the page looks full.
- [ ] Check the README renders: badges, four screenshots, tables.
- [ ] **Pin** the repository on your GitHub profile.
- [ ] Open **Backend/.env.example** on GitHub and confirm it holds placeholders only.

**Remember.** Add the **Website** field and the live demo link in the README only after job 2.

**Done when:** the repository page has a description, topics, a preview image, and every image in the README loads.

## 2. Deploy the app

**Goal.** Anyone can open a link, sign up, and place an order. The frontend and backend live on different hosts, as planned.

| Part | Host | Folder | Key settings |
|---|---|---|---|
| **Database** | MongoDB Atlas (already) | - | Network Access must allow the backend host (0.0.0.0/0 is the simplest for a demo) |
| **Backend** | Render or Railway | Backend | Build: npm install. Start: node server.js. Env: MONGODB_URI, JWT_SECRET, CLIENT_ORIGIN, NODE_ENV=production |
| **Frontend** | Vercel or Netlify | Frontend | Framework: Vite. Build: npm run build. Output: dist. Env: VITE_API_URL |

### Steps

1. In **Backend/package.json** add the script **"start": "node server.js"**. Hosts run it in production (nodemon is for development only).
2. In **server.js** add **app.set("trust proxy", 1)** before the rate limiters, so they see real visitor IPs, not the host's.
3. Deploy the **backend** first. Add the environment variables in the host's dashboard (never in code). Open **/api/health** on the live address; it should return {ok:true}.
4. Deploy the **frontend** with the root directory set to **Frontend**. The SPA rules are already in the project (vercel.json and public/_redirects), so /orders survives a refresh.
5. **Make cookies work across the two addresses.** Add a rule on the frontend host that forwards /api/* to the backend (a rewrite in vercel.json, or a line in _redirects on Netlify). Then the browser talks to one address only, the login cookie is first-party, and browsers that block third-party cookies still work.
6. Set **CLIENT_ORIGIN** on the backend to the live frontend address, and redeploy.
7. Test on your phone: sign up, log in, order, refresh on /orders, log out.
8. Put the live link in the README and in the GitHub About box.

> **Be honest about.** **Free backends sleep** after some minutes without traffic, so the first request can take 30 to 60 seconds. Your loading spinners already handle this. Mention it in the README so reviewers do not think the app is broken.

> **Remember.** Use a **separate database** (or at least a separate database name) for production once real people use it, so test orders never mix with real ones.

**Done when:** the full flow works from the live URL on a phone, and the README links to it.

# 3. Password reset by email

**Goal.** A user who forgot their password, or got locked out, can recover the account safely.

### The flow

1. User clicks **Forgot password?** on the login page and enters an email.
2. **POST /api/auth/forgot-password**: always answers "If that email exists, we sent a link" (same answer whether or not the account exists, so nobody can discover accounts).
3. If the account exists, the server creates a **random token** (crypto.randomBytes), stores only its **SHA-256 hash** and an expiry (about 30 minutes) on the user, and emails a link: **/reset-password?token=...**
4. The user opens the link and types a new password (same rules as signup).
5. **POST /api/auth/reset-password**: finds the user by the token hash, checks the expiry, hashes the new password with bcrypt, **deletes the token**, and resets failedLoginAttempts and

lockUntil.

6. The user logs in with the new password.

### What to build

| Where | What |
|---|---|
| `User model` | Two new fields: resetTokenHash and resetTokenExpires (both select:false) |
| `authController` | forgotPassword and resetPassword, validated with Zod |
| `authRoutes` | Two new routes; the forgot route gets its own strict rate limiter |
| `Email sending` | Nodemailer with a free provider (Resend, Brevo or Gmail SMTP). In development use Mailtrap or Ethereal so no real emails go out |
| `Frontend` | ForgotPassword page, ResetPassword page, and a link on the login page. Update the lockout message to point to the reset page |

> **Be honest about.** **Never store the raw token** in the database, and never reveal whether an email has an account. Tokens must be single-use and expire. A token sitting in the database in plain text is as bad as a stored password.

**Done when:** you can lock an account, reset the password through the emailed link, and log in; the same link fails the second time and after it expires.

# 4. Tests and CI

**Goal.** The important rules are checked by code, not by clicking. Every push is checked automatically.

| Layer | Tools | What to test first |
|---|---|---|
| **Backend unit** | Vitest or Jest | priceBill: PLATTER50 caps at 100 and only for the first 3 orders, FREESHIP, free delivery at 299 and above, GST rounding |
| **Backend API** | Supertest + mongodb-memory-server | Signup (409 for a repeat email), login, lockout after 5 wrong passwords (429), requireAuth (401), orders ignore client-sent prices, another user's order returns 404, invalid ids return 400 |
| **Frontend unit** | Vitest + React Testing Library | Card number check (Luhn), expiry check, cart add/remove and the switch-restaurant prompt, login form errors |
| **End to end** | Playwright | Sign up, add a dish, check out, see the order on My orders |
| **CI** | GitHub Actions | Run all of the above on every push, and show a green badge in the README |

### Steps

1. Split the backend into **app.js** (creates and exports the Express app) and **server.js** (connects to the database and calls listen). Tests import the app without starting a server.
2. Add **mongodb-memory-server** so tests use a temporary database, never your Atlas data.
3. Write the pricing tests first; they are short and catch real bugs.
4. Write the auth and order API tests (the lockout and ownership tests are the most impressive).
5. Add frontend tests for the pure logic (validation, cart), then one Playwright flow.
6. Add **.github/workflows/ci.yml** that installs, tests and builds both folders.

> **Say it like this.** When asked **"how do you test?"**, say: "Unit tests for the pricing and validation rules, API tests with Supertest against an in-memory database for auth, lockout and order ownership, and one Playwright test for the main purchase flow, all run in GitHub Actions."

**Done when: npm test** passes in both folders and the CI badge on GitHub is green.

# 5. Real payments (Razorpay, test mode)

**Goal.** Customers can pay online, and your server only marks an order as paid after Razorpay proves it.

### The flow

1. Customer chooses an online method and clicks **Place order**.
2. The server **creates the order** as usual (prices from the database) with paymentStatus "pending", then asks Razorpay to create a payment order for that total (in paise).
3. The frontend opens **Razorpay Checkout**, their hosted form. Card, UPI and bank details go to Razorpay, **never to your server**.
4. On success Razorpay returns **payment_id, order_id and signature**.
5. The frontend sends those three to **POST /api/payments/verify**.
6. The server recomputes the signature with **HMAC-SHA256** and the secret key. Only if it matches does it set paymentStatus to "paid".
7. Optional but best practice: a **webhook** as a backup, in case the customer closes the tab after paying.

### What changes in the project

| Where | What |
|---|---|
| `Order model` | paymentStatus (pending, paid, failed), razorpayOrderId, razorpayPaymentId |
| `New routes` | POST /api/payments/create and POST /api/payments/verify, both behind requireAuth |
| `Environment` | RAZORPAY_KEY_ID (can be public) and RAZORPAY_KEY_SECRET (server only, never in the frontend) |
| `Frontend` | Replace the demo card, UPI and net banking form with the Razorpay popup. Keep cash on delivery as it is |
| `Orders page` | Show a Paid or Payment pending badge |

> **Be honest about.** **Never trust the browser** saying "payment succeeded". Only the signature check on your server (or a webhook) can mark an order paid. Test mode needs only a Razorpay account and test keys, with no real money involved.

**Done when:** a test payment marks the order Paid, and tampering with the signature leaves it Pending.

# 6. Polish (pick two or three)

These are optional. Each one is a small, visible improvement that shows care.

| Idea | What it involves |
|---|---|
| **Check the address city** | Server rejects an order whose delivery address does not match the restaurant's city |
| **Cancel an order** | A Cancel button while the order is Pending; the server enforces it (never only the button) |

| Idea | What it involves |
|---|---|
| **Order detail page** | A page per order using GET /api/orders/:id, linked from My orders |
| **Page titles** | Set document.title on each page so tabs and shared links read well |
| **Error boundary** | A friendly fallback screen if a component crashes, instead of a blank page |
| **Paginate orders** | Load 10 orders at a time with a Load more button |
| **Accessibility and speed** | Run Lighthouse; compress large images; check keyboard use on every page |
| **Dark mode** | Colour tokens already exist in CSS variables, so this is mostly a toggle and a second palette |

# Your tracking list

Tick these off as you go.

- [ ] **1.** GitHub About, topics, preview image, pinned repo
- [ ] **2.** Backend deployed and /api/health works
- [ ] **2.** Frontend deployed, /api forwarded, full flow works on a phone
- [ ] **2.** Live link added to README and About
- [ ] **4.** Pricing and auth tests pass
- [ ] **4.** Order ownership and lockout tests pass
- [ ] **4.** Playwright flow passes, CI badge is green
- [ ] **3.** Forgot and reset password working end to end
- [ ] **5.** Razorpay test payment marks an order Paid
- [ ] **6.** Two or three polish items done

> **Remember.** Work one job at a time, and finish each one (tested and pushed to GitHub) before starting the next. A smaller project that is finished and deployed is worth more than a bigger one that is half done.